orbit · a spec-driven workflow for AI development

Orbit Spec Introduction

Part 1

How to use orbit

The loop, step by step: architect, explore, propose, review, apply, review, archive, then architect again to ratify. At each step, what the AI produces and what the user decides.

Part 2

Case study: overtime

A retirement model for one household, rebuilt from an AI-written proof of concept. One program, ten stages, twenty changes in about a month. Part 1's "In overtime" examples come from here.

Part 3

Case study: orbit on itself

orbit's own trail showed its review cycle producing its own findings, and an orbit program fixed it. The blind pair, the severity ladder and the size ceiling came from this.

Note: orbit runs in Claude Code today; other AI tools are used only as external reviewers. Running orbit in them is future work.

Sal · October 2026 · github.com/las-sal/orbit-spec

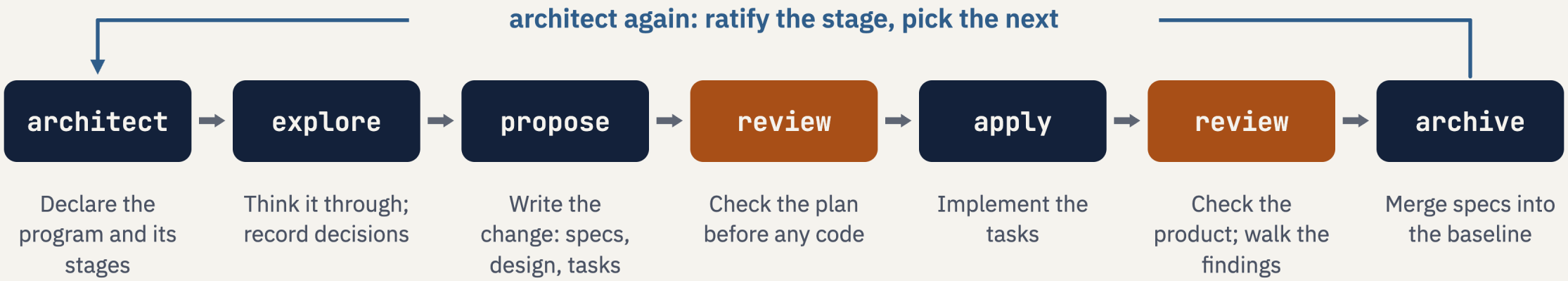
Part 1

Orbit Spec Details

One loop with eight steps. At each one, what the AI produces and what the user decides.

THE LOOP

One loop, eight steps



Orange = a reviewed gate: apply waits on the proposal review, archive on the system review. An architect session is suggested, never required; a one-off fix can start at explore.

Source: orbit docs/workflows.md

STEP 1 OF 8

Break a complex problem into stages

```
/orbit:architect <program>
```

What the AI does

Drafts the architecture doc with the user: goals written so they can fail, rulings, and stages small enough to explore, each with a charter, dependencies and exit criteria. Each stage's state comes from what's actually in the repo, never from a status someone typed.

What the user does

Does the decomposition with the AI, and owns the goals, the stage boundaries and every ruling. Each ruling is dated and names who made it.

In overtime

One program, *overtime_creation*: five goals, 34 rulings, and ten stages that each answered one part of the problem.

In orbit

clean-up-review-cycle opened on Sep 11 from D6's trail. Two sessions that day measured why reviews were getting costlier, then re-cut the plan into twelve small stages in four phases.

ARCHITECT IN PRACTICE

Overtime: One hard problem, ten stages

THE PROBLEM

An AI-built proof of concept that had lost focus: deep on one question, private data in code, three of four questions drifted away.

1 · Judge what exists

`salvage-audit` `data-review`

2 · Lay the foundation

`repo-foundation` `profile-schema` `ingestion` `bridge-retirement`

3 · Answer the question

`plan-core` `document-builder`

4 · Deliver and deepen

`delivery-surface` `model-depth`

EVERY STAGE CARRIES THE SAME FOUR PARTS

`Depends-on`
what must land first

`Charter`
what this stage is for

`Exit criteria`
what “done” means, checkable

`Complete [x]`
dated, ratified by the user

Source: overtime orbit/architecture/overtime_creation.md

STEP 2 OF 8

Think each stage through

```
/orbit:explore
```

What the AI does

Reads the code, asks questions and compares options, and changes nothing. It writes *explore.md*: decisions (D1, D2…), open questions and “considered & out”.

What the user does

Argues with it and sizes the work. If a stage is too big to propose and apply as one change, the user splits it. Decisions that bind the program become rulings, and architect builds the split into the overall program.

In overtime

model-depth ran seven explorations. The seventh ruled out a reduced 401(k) deferral, recorded under Considered & out with a GitHub issue.

EXPLORE → BACK TO ARCHITECT → PROPOSE

Too big? Re-cut it before you propose

`profile-schema`
as first declared: one big stage

→

`profile-schema`
the input contracts and schema

`ingestion`
reading the three inputs

`bridge-retirement`
the engine reads what ingestion wrote

Explore surfaced it on Sep 14. It went back to architect, so the split lives in the plan as stages the position readout can see, not as prose inside one stage. Its nine exit criteria were redistributed.

Bigger changes cost more review

On orbit itself, every stage touching more than 5 capabilities cost **12–17 review passes**. The one that converged in a week touched 2. Now anything over 3 is split before review.

A re-cut takes many shapes

Split: *profile-schema* became three stages. **Move:** an exit criterion went from *repo-foundation* to *profile-schema*. **Merge or cut:** orbit's own re-cut did both. **Fold out:** *generalise* went to an issue.

Source: overtime_creation.md status log, 2026-09-14; orbit clean-up-review-cycle.md, R-crc-1

STEP 3 OF 8

Write the change

```
/orbit:propose
```

What the AI does

Writes the change folder: *proposal.md* (why, what changes, capabilities), delta specs (requirements with scenarios), *design.md* and *tasks.md*. Optionally opens a draft PR.

What the user does

Reads the proposal before any code exists. It's the cheapest point to change course, and the delta specs are what reviewers will hold the code to.

In overtime

model-depth-monte-carlo added two capabilities and modified twelve, each named in the proposal with what changes.

architect | explore | propose | **review** | apply | review | archive | ratify

STEP 4 OF 8 · GATE

Review the plan

The gate needs two reviews, in order. It always opens with a **blind pair**: two independent reviewers from different models. It closes with the **in-context** review, which knows the conversation and checks the fixes.

The gate passes when both slots are covered and every CRITICAL and WARNING is resolved by a walk.

1

Blind pair

`/orbit:review --as proposal --blind-pair`

A fresh and an external review of the same state, neither seeing the other. Covers slot 1 once both land.

2

Walk

`/orbit:address-reviews`

Every finding from both halves, one at a time, each checked against the files first.

3

In-context, then walk

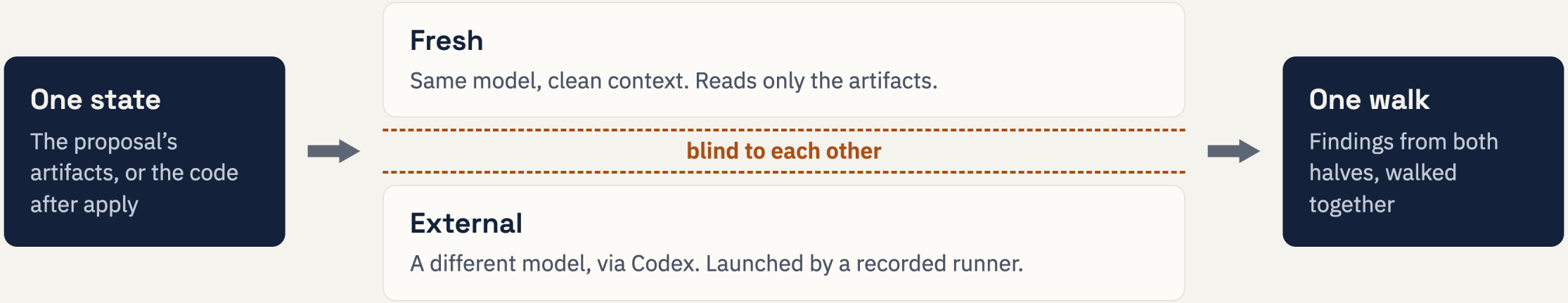
`/orbit:review --as proposal`

My working session confirms the walk’s fixes landed. Covers slot 2.

Source: orbit docs/workflows.md, docs/concepts.md

THE BLIND PAIR

Two reviewers, two models, no peeking



Different models, different finds

In overtime, 40 external returns raised 56 CRITICALs; 85 internal passes raised 38.

Disagreement is evidence

Where the two halves disagree, the disagreement itself is worth walking.

One command

The fresh half runs here; the external half launches on its own and is filed on the trail.

Source: orbit docs/concepts.md; overtime trail counts approximate across record formats

STEP 5 OF 8

Build it

```
/orbit:apply
```

What the AI does

Implements the tasks in *tasks.md* and checks them off, each session recorded on the trail. The apply gate passes when every task is checked; tasks left for the user don't hold it.

What the user does

Invokes it and stays reachable. Running apply before the proposal gate passes is allowed: orbit says so, asks, and records the decision on the trail.

In overtime

model-depth-monte-carlo's apply ran across three sessions, each one recorded in the change's *.orbit-runs/* trail.

STEP 6 OF 8 · GATE

Review the product, walk the findings

```
/orbit:review --as system --blind-pair
```

```
/orbit:address-reviews
```

The first pass checks the code against the specs; the rest look system-wide. Then the walk takes findings one at a time, each checked against the current tree first.

Obvious fix

Previewed, then applied.

Decision

Briefed in prose with options. The user ratifies one.

Stale

Already untrue. Suppressed with evidence, never “fixed” twice.

STEP 7 OF 8

Record it

```
/orbit:archive
```

What the AI does

Merges the delta specs into the baseline in *orbit/specs/*, moves the change and its whole review trail to the archive, and marks a draft PR ready.

What the user does

Confirms the system gate and answers any tasks left for the user. Then the baseline says what the system now does, not what was once planned.

In overtime

20 changes archived into 30 baseline capabilities. Each change fingerprints the requirements it touches, so two changes never silently overwrite each other.

STEP 8 OF 8

Ratify the stage

```
/orbit:architect <program>
```

What the AI does

Reads the stage's exit criteria against what actually landed and reports on each one. The stage completes only when its change is archived and the user ratifies it.

What the user does

Ratifies the stage, or declares it ended without meeting its criteria. Rulings that came out of the work get written down, including the rejected alternatives.

In overtime

The closing session on 2026-10-12 moved two stages out to GitHub issue #22 and read all five goals as met, with scope notes.

FINDINGS

Severity means consequence

CRITICAL	A user path breaks or data is lost, **and** the finding carries a reproduction someone can re-run.
WARNING	An implementation built to the artifact as written could diverge from what it intends.
SUGGESTION	Everything else. Suggestions never hold a gate.

A gate passes only when every CRITICAL and WARNING is resolved by a walk.

Source: orbit-review spec

GATES AND CONSENT

User invokes every step (for now)

```
$ orbit status
review-proposal passed
apply ready
review-system waiting on apply

Ready when you are: /orbit:apply
```

Illustrative readout

Passed means eligible

A gate passing makes the next step available. It does not run it.

The AI recommends

It names the next step and never runs it unasked. An AI-initiated apply or archive asks first.

Overrides are recorded

Going past an open gate is allowed, and the trail records the gate's state when the user did.

Source: orbit docs/workflows.md, docs/cli.md

ORBIT STATUS

Where things stand, and how to resume

```
$ orbit status
model-depth-monte-carlo (stage model-depth · overtime_creation)
explore ✓ → propose ✓ → review ⟲3 ✓ → apply ✓ 33/33 → review ⟲ → archive
last: apply — 33 of 33 tasks done.
Ready when you are: /orbit:review --as system --reviewer blind-pair …
```

```
$ orbit status --change model-depth-monte-carlo --boot
Read in order: proposal · design · specs · tasks
Latest emits: explore · propose · review(proposal)
              address-reviews · apply
              each with path + one-line headline
Touchpoints: 19 files predicted by design.md
Ready when you are: /orbit:review --as system …
```

Know your position

The strip, the last step, and the one command eligible next. Computed from files, never stored.

Boot a clean context

--boot gives a new session ordered pointers into the change. No handoff file to maintain.

Replayed: overtime’s model-depth-monte-carlo with its trail cut after the last apply, through orbit v2.0.1 · output condensed

ORBIT BUILT WITH ORBIT

Every review feature was itself an orbit change

998
commits over 72 days, Jun 5 to Oct 16

59
archived changes

31
baseline capabilities, each with a spec

3,747
tests in 177 files, all passing

123
external reviews returned (GPT via Codex)

1,505
findings resolved across 373 walks

Measured 2026-10-16 at main 6101190 · trail counts approximate across record formats

Part 2

overtime: orbit in use

A retirement model rebuilt from an AI-written proof of concept: one program, ten stages, twenty changes, Sep 10 to Oct 12.

WHAT OVERTIME IS

One household’s retirement model

Rebuilt from a proof of concept into a program that answers **whether the plan works**, not just how to convert.

A year-by-year engine

Mortality, survivorship and an estate layer in every analysis.

One recommender

Social Security claim ages searched together with the Roth conversion plan and the 401(k) levers.

10,000 seeded market histories

How often the plan stays solvent, and which alternatives beat it.

Tax law as data

29 statutory rules, each with a dated citation and a worked case the engine reproduces.

Private by construction: real data lives in a gitignored *user_data/*, and a privacy scan refuses any commit carrying a household value.

WHY ORBIT WAS BROUGHT IN

“Adversarial review optimises what is in front of it and has no opinion about what is missing.”

The proof of concept was built over fifteen versions and ten review rounds. Three of its four starting questions drifted off the page without ever being rejected.

The standing question list

Every review checks the work against it. A question is answered or ruled out on the record, never dropped.

1	Does the plan work? What fails first?	**Answered**
2	Whether and when to convert to Roth?	**Answered**
3	How to manage the pre-tax 401(k)?	**Answered (in 6 parts)**
4	Is liquidity adequate to the horizon?	**Answered**
5	What software should complement this?	**Ruled out → issue #23**
6	Can someone else use it?	**Ruled out → issue #22**
7	What does relocating change?	**Ruled out → issue #22**

Source: overtime orbit/architecture/overtime_creation.md

THE PROGRAM'S GOALS

Each goal is written so it can fail

G0

Model the tax code correctly

Fails if any rule lacks a dated citation or a worked case the engine reproduces.

Met, for Pennsylvania; 4 rules flagged as simplified

G1

A meaningful plan

Fails if the user can't answer "does the plan work, and what are the levers?".

Met

G2

Generic

Fails if any household value or assumption lives in code.

Met at its minimum: the fictional example runs end to end

G3

Private by construction

Fails if a scan finds a household value in the repo; it gates commits.

Met

G4

Sensitivity by profile edit

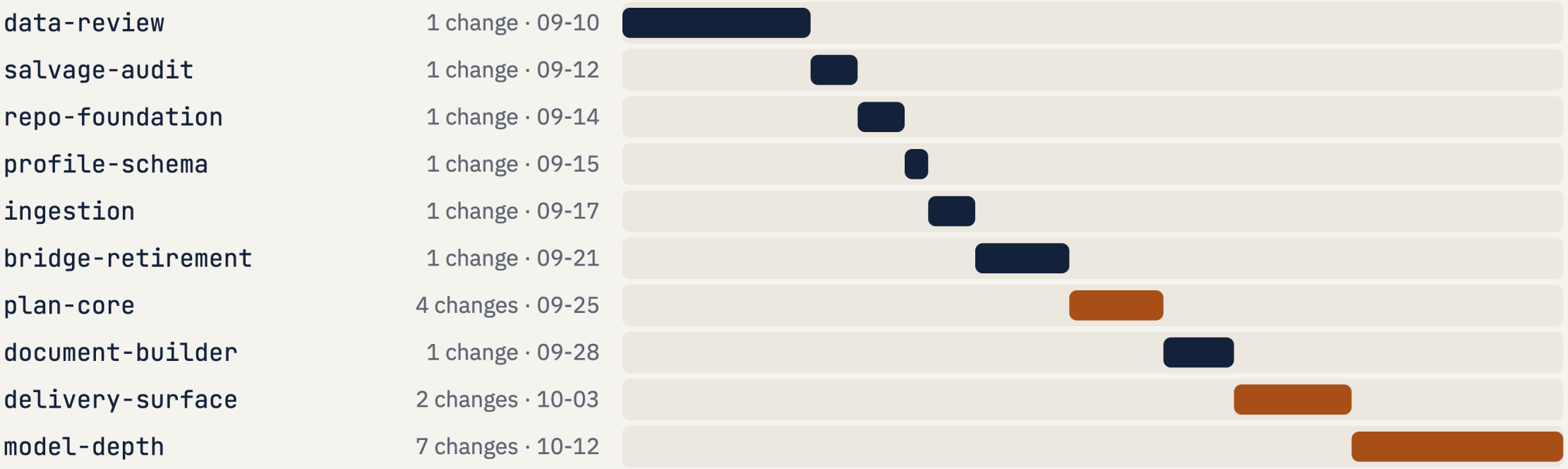
Fails if changing one input needs an engine edit.

Met

Source: overtime_creation.md, closing reading 2026-10-12

FROM SEP 2 TO OCT 12

Ten stages, all ratified



Each bar runs from the previous stage's ratification to this one's. **Orange** stages took more than one change.

Source: overtime_creation.md stage records · axis Sep 2 to Oct 12

OVERTIME BY THE NUMBERS

One program, a complete trail

170
commits over 25 days, Sep 10 to Oct 12

20
archived changes across 10 stages

30
baseline capabilities

29
tax rules, each cited with a worked case

~1,118
test functions in 73 files

516
run-summary records on the trail

40
external reviews returned

126
walk logs

Measured 2026-10-16 at main 3a91f84 · trail counts approximate across record formats

THE TRAIL

Every step leaves a record

```
model-depth-monte-carlo/.orbit-runs/
  Oct 4 17:07  explore ×2
        18:08  propose
        18:19  review · proposal fresh
        18:29  walk
        18:44  review · proposal external
        19:01  walk
        19:06  review · proposal in-context
        19:11  walk
        20:47  apply ×3, to 23:04
  Oct 5 00:03  review · system external
        00:16  review · system fresh
        00:37  walk ×2, incl. the CRITICAL
        00:56  review · system in-context
        01:16  walk
        01:31  archive
```

23 records, about 8½ hours from first exploration to archive

What it covers

Every workflow run on a change writes one record: explore, propose, each review, each walk, apply, archive. Architect sessions and drift audits keep trails of their own.

How it works

The AI writes only the narrative and its findings. *orbit emit* composes the rest (timestamps, counts, gate state, next step) and refuses a record that doesn't hold together.

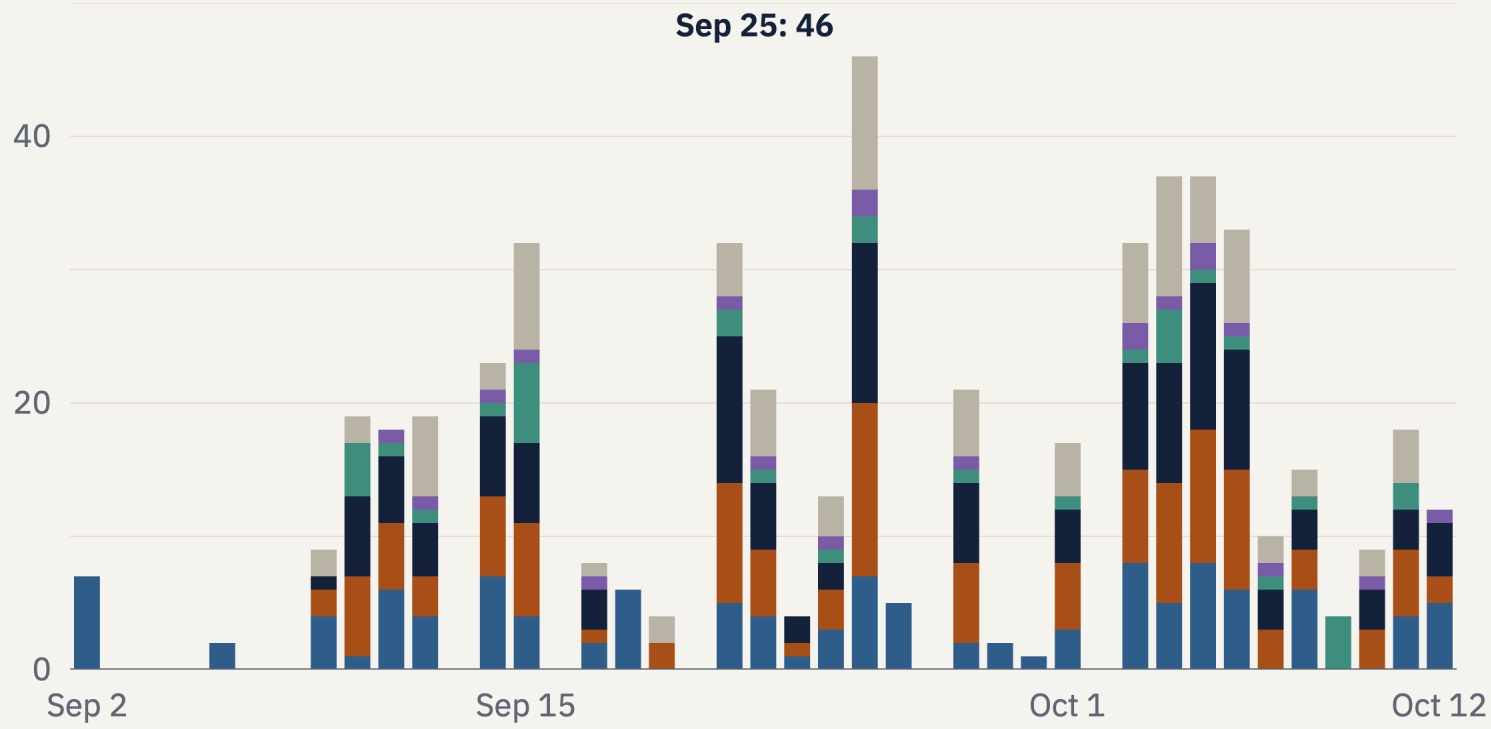
Why it matters

Gates are computed from it, never asserted. It moves with the change into the archive. And it is replayable: the charts that follow are a script over these files.

Source: overtime model-depth-monte-carlo trail; orbit docs/concepts.md, docs/workflow-engine.md

OVERTIME'S TRAIL

516 orbit records, day by day



Decide 118
architect, explore, propose

Review 125
proposal, system and external returns

Walk 126
address-reviews

Build 36
apply, verify

Record 20
archive

Other 91
drift audits, external requests

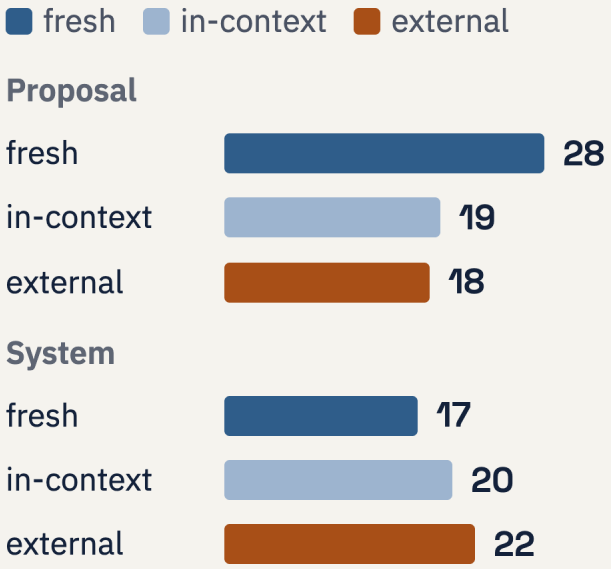
Source: every file in overtime's .orbit-runs trails, by filename date and kind

OVERTIME'S TRAIL

Reviews, findings, walks

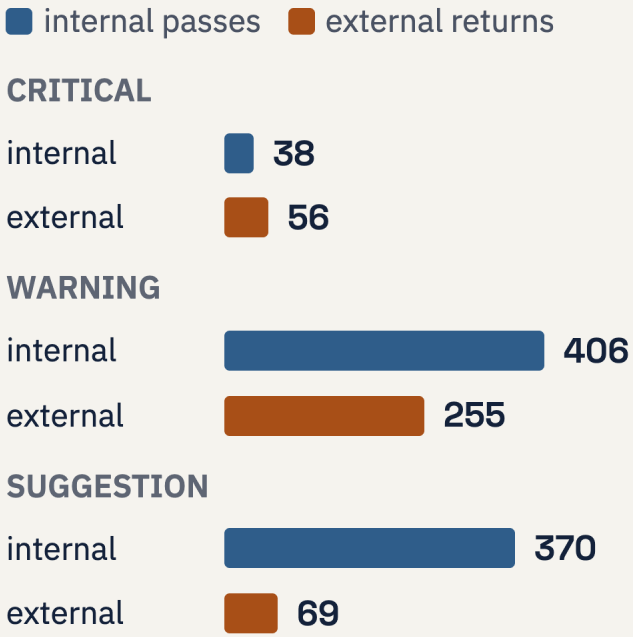
1 · REVIEWS RUN

By mode and reviewer



2 · FINDINGS RAISED

By severity and source

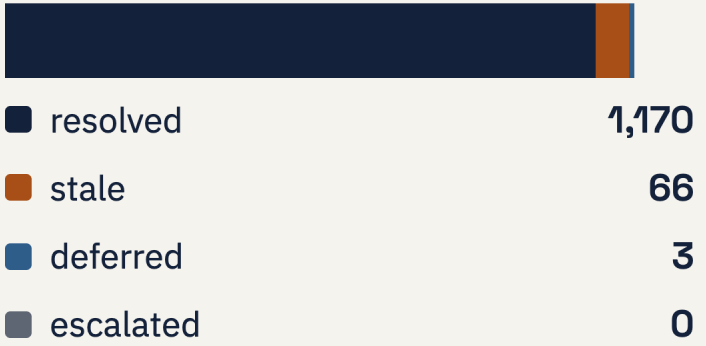


3 · HOW WALKS ENDED

Outcomes

94%

of walked findings were resolved; the rest were mostly stale, suppressed with evidence.

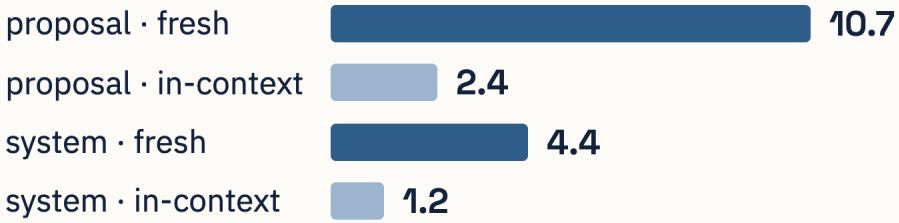


across 126 walk logs

Source: overtime .orbit-runs trails · approximate across record formats

Four patterns in the trail

Each gate converges



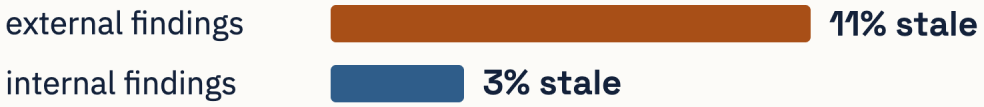
CRITICAL + WARNING per internal pass. The confirming in-context pass finds about a quarter as much.

Most findings never need the user



Of 1,216 itemized walk resolutions. But 37% of CRITICALs came to the user as decisions.

Pushback matters most on external findings



45 of 405 external findings were already untrue when walked, against 25 of 807 internal.

The cadence got faster



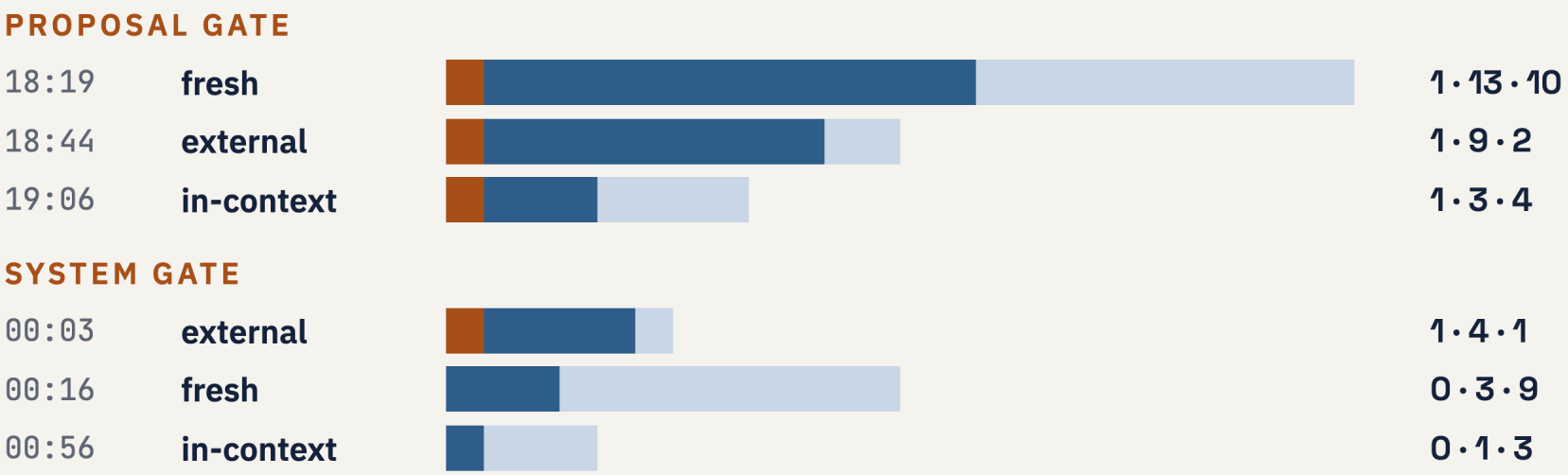
Median time from a change's first record to its archive. Records per change settled near 22.

Source: overtime .orbit-runs · in-context passes run after a walk, so part of the drop is fixes landing · lead time is wall clock

AN EXAMPLE

One change, six reviews, converging

CRITICAL WARNING SUGGESTION a walk follows every review



CRITICAL + WARNING per review: **14** → **10** → **4** on the plan, then **5** → **3** → **1** on the code.

Source: overtime model-depth-monte-carlo trail · internal counts from review scorecards, external from returned reports

A WALK, UP CLOSE

One CRITICAL, from finding to fix

CRITICAL A frozen market snapshot can change without invalidating the run

1 · Raised

External system review (GPT-6) on *model-depth-monte-carlo*, Oct 5: six findings, one CRITICAL.

2 · Verified

All 6 checked against the current tree before anything changed. None were stale.

3 · Decided

Classified as a decision. The user picked option A in chat: refuse any run whose snapshot no longer matches the manifest's digest.

4 · Rippled

Two delta specs, the design, three test files and the skill updated. 830 unit tests passed after the walk.

Source: overtime model-depth-monte-carlo .orbit-runs/address-reviews-2026-10-05T00-37-44Z.json

PUSHBACK

A CRITICAL that was already fixed

bridge-retirement · external review · Sep 21

“The omitted derived-profile delta leaves two normative contradictions”

The walk re-checked the tree: an earlier walk had already added the missing block to the delta spec. The finding was suppressed as **stale, with that evidence**, instead of being “fixed” a second time.

1,170
findings resolved

66
stale, suppressed with evidence

3
deferred · 0 escalated

Source: overtime bridge-retirement walk logs; totals across all 126 walk logs

THE CADENCE, BEFORE AND AFTER

Practised by hand, then built in

OVERTIME · BY HAND, FROM SEP 24

13 of 13 changes ran a fresh and an external pass, twice

Once on the proposal, once on the system: a blind pair, run as two steps. Earlier changes varied.

ORBIT V2.0.X · BUILT IN

`/orbit:review --as system --blind-pair`

One command, both halves blind to each other. A recorded runner launches the external half with no copy and paste.

overtime ran an earlier orbit. It shows the method being practised while it was designed; it did not use the v2.0.1 commands.

Source: overtime .orbit-runs trails; orbit docs/commands.md

WHAT THE TRAIL SHOWS

A different model finds different problems

Severity raised	Internal passes (85)	External returns (40)
CRITICAL	38	56
WARNING	406	255
SUGGESTION	370	69

40 external returns raised more CRITICALs than 85 internal passes.

These are findings raised, not defects shipped. Almost all were resolved or found stale before archive.

Source: script over every overtime .orbit-runs file · approximate across record formats

HONEST NOTES

What this case study does and doesn’t show

An older orbit

overtime predates the blind-pair command, the external runner and v2.0’s renames. It shows the method, run by hand.

Approximate counts

Trail counts come from a script across several record formats. A finding raised by both halves of a pair counts twice.

Raised, not shipped

Severity counts are what reviewers raised. The walk outcomes say how each one ended.

A bounded model

Pennsylvania only, four rules flagged as simplified, one household plus a fictional example. Questions 5 to 7 live in issues.

Part 3

orbit on itself

clean-up-review-cycle: the trail diagnosed orbit's own review cycle, and an orbit program fixed it.

CONTEXT

orbit is built with orbit: five programs

Jun 5 – Jul 7

fork-and-orbify

Forked OpenSpec 1.3.1 into one CLI, renamed it orbit, added engine-computed gates.

8 changes

`c1, c2 …`

Jul – Sep 11

D-arc

Consent, the architect tier, orientation, review consistency, context economy, PR lifecycle.

6 changes

`d1 – d6`

Sep 11 – Oct 12

clean-up-review-cycle

Made the review cycle converge. The subject of Part 3.

35 changes

`rc01 – rc18`

Oct 12 – 13

update-gh-documentation

Rewrote the docs under test. orbit went public as v2.0.0.

5 changes

Oct 13 – 16

refine-documentation-and-architect

Architect-first, a CLI that says orbit, and v2.0.1.

4 changes

Each program is an architecture doc whose stages became changes, 59 in all. Change names carry their program: c1 comes from the fork, d2–d6 from the D-arc, rc01–rc18 from this program.

Source: orbit architecture status and orbit/architecture/*.md · 58 changes in programs, 1 outside

HOW REVIEW WORKED IN SEPTEMBER

A loop with no exit



The gate rule then

Repeat independent passes until one comes back with no CRITICAL and every finding resolved.

Why it couldn’t end

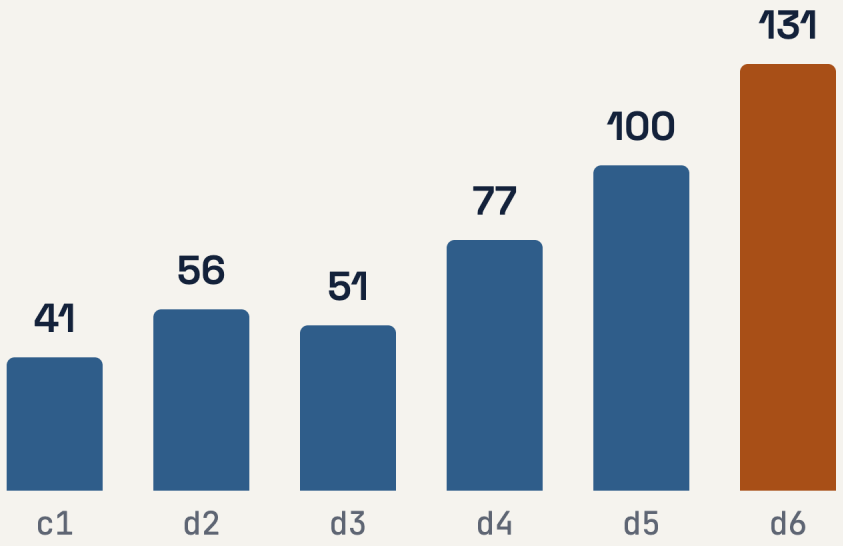
Every walk changed the files, so the last pass no longer covered them. The next pass read the newest edits first and found fault there.

Source: clean-up-review-cycle.md, Premise

THE DIAGNOSIS

The cycle was producing its own findings

FINDINGS PER CHANGE, D-ARC



Each over 12 to 17 passes. D6 walked 42 system findings to zero and **still archived through an override**.

Later passes landed 60–100% of their findings on files the previous walk had just edited.

Every walk forced a new pass
A walk edits files; the edit stales the last review; the next pass finds the walk's edits.

Prompts aimed at the fixed areas
"Resolved since last review" spent fresh eyes on the newest tenth of the work.

WARNING had no bound
Prose-heavy artifacts gave a diligent reviewer unlimited defensible WARNINGs.

Source: orbit orbit/architecture/clean-up-review-cycle.md, Premise (measured 2026-09-11)

THE PROGRAM

clean-up-review-cycle, at a glance

Make the review cycle cheaper by making it produce fewer of its own findings.

Sep 11 – Oct 12	40	35	4
one month	stages, 8 ended or split	changes archived	phases, ordered on purpose

GOALS, EACH WRITTEN SO IT CAN FAIL

- **G0** Measured on its own trail: three stages in a row converge under G1.
- **G1** The cycle ends in agreement, recorded as a readiness judgement, not an override.
- **G2** Claims a machine can check leave the reviewer’s budget.
- **G3** Less is injected upstream: walks sweep whole requirements, proposals stay small.
- **G4** The engine owns the steps that are deterministic.

Source: clean-up-review-cycle.md, Premise, Goals and architecture status

THE RE-CUT

Small stages, ordered on purpose

“Break it into small approachable chunks and be intentional about how they are ordered.” Sal, Sep 11

Over 5 capabilities

Every such stage cost **12–17 review passes**.

2 capabilities, 9 requirements

c2-parallel-merge, the one design-heavy stage that converged in a week.

`R-crc-1` **At most 3 capabilities, ~12 requirements, under 20 tasks. Over the ceiling, split before review.**

Source: clean-up-review-cycle.md, R-crc-1 and status log 2026-09-11

WHERE THE ROUTINE CAME FROM

Blind pair, walk, then in-context

Sep 11 — **Cohesion review**
Recommends one canonical cadence: two blind reviews before any walk, one merged walk, one verification pass.

Sep 21 — **rc04a**
Archives carry a readiness judgement formed against named passes.

Oct 3 — **rc02b**
The severity ladder: a CRITICAL needs a reproduction.

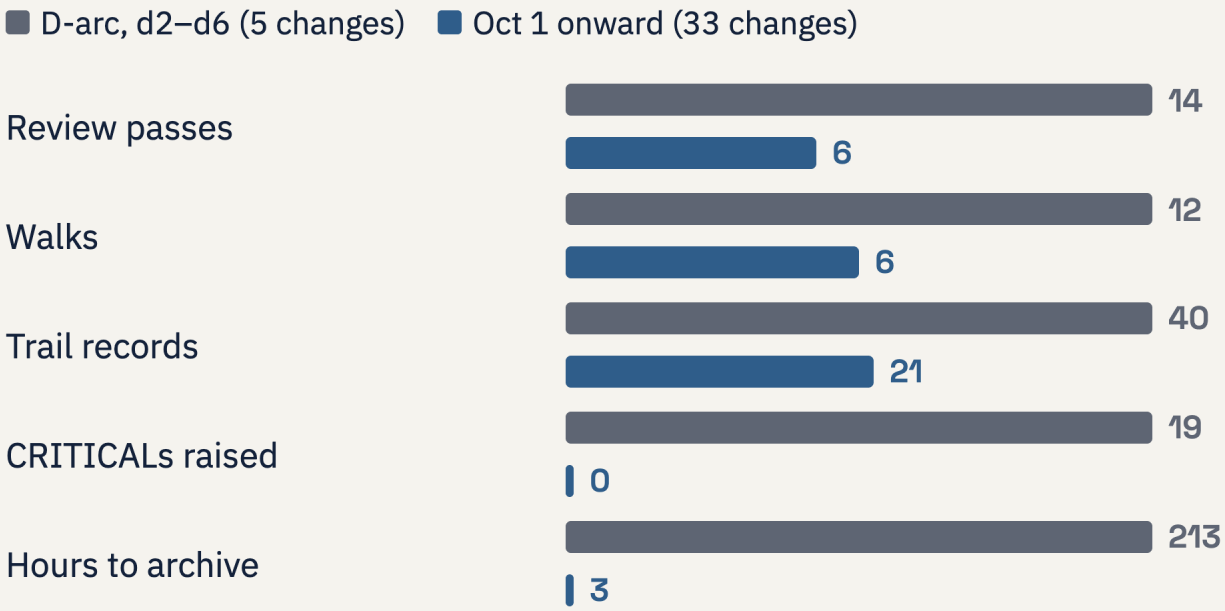
Oct 6 — **rc17d · rc17b**
A recorded runner launches the external half; the blind pair becomes one command.

Oct 8 — **rc18**
A passed gate with an unwalked pass names the walk next.

Source: clean-up-review-cycle.md status log and stages; archived change dates

BEFORE AND AFTER

Medians per change, from orbit's own trail



HOW EACH OF 59 CHANGES ARCHIVED, IN ORDER



Computed from every orbit .orbit-runs trail · medians · change mix differs between periods

HONEST GRADING

Two goals failed, and the record says so

G0	**Converge on its own trail**	**Met**	Nine stages in a row passed a computed gate.
G1	**End in agreement**	**Met, fired once**	One review grew a proposal; recorded as fired.
G2	**Checks leave review**	**Falsified**	An unmarked count still reached a reviewer.
G3	**Inject less upstream**	**Falsified**	16 ripple misses; 10 searches missed in-scope sites.
G4	**Engine owns deterministic steps**	**Met**	Graded at rc12’s ratification.

> “Failing in a new way is the product; failing the same way twice is the signal to codify.”

Source: clean-up-review-cycle.md status log, goals graded 2026-10-12

TRY IT

Start with the program, not the code.

```
npm install -g github:las-sal/orbit-spec#v2.0.1
```

```
/orbit:architect <program> · or /orbit:explore for a one-off fix
```

```
orbit status · where things stand, what's next
```

github.com/las-sal/orbit-spec · docs/getting-started.md